



Python Nivel intermedio - EPC VVG04.

MANEJO DE ERRORES Y DEPURACIÓN.

UNIDAD 4.

4A. Ejercicio. Validación de datos

ALUMNO: HURTADO LÓPEZ PEDRO.

PROFESOR: ORTEGA FIGUEROA EFRÉN
ALEJANDRO.

Domingo 29 de marzo del 2026.

Introducción.

Este sencillo ejercicio práctico está diseñado para consolidar los conceptos básicos del manejo de errores, el uso de las excepciones que hemos estudiado en la Unidad 4. Las excepciones que vimos en el ejercicio son las siguientes:

- **ZeroDivisionError** -> División por cero.
- **ValueError** -> Conversión inválida de tipos de datos.
- **TypeError** -> Operación entre tipos incompatibles.

Recordando la estructura de las sentencias que tiene Python para que evitar que un programa se detenga bruscamente cuando ocurre un error:

- **try**: Contiene el bloque de código que podría generar una excepción.
- **except**: Define qué hacer si ocurre un error específico.
- **finally**: Se ejecuta siempre, ocurra o no una excepción, y se usa comúnmente para cerrar archivos, liberar recursos o mostrar mensajes finales. Es un bloque opcional.

Ejercicio 1. Planteamiento del problema.

Sistema para el Registro de Usuarios: Validador de Datos.

Desarrollar una pequeña aplicación para el registro de usuarios para guardar nombre, edad y correo electrónico. El sistema debe validar los datos introducidos y manejar los errores al ingresar información incorrecta:

1. Crear un archivo llamado **registro.py**:



The screenshot shows a file explorer on the left with the following structure:

- ▼ Ejercicio_U4_Validación_de_Datos
 - Ejercicio_U4.py
 - registro.py

On the right, the beginning of the `registro.py` file is shown:

```
Ejercicio_U4_Validación_de_Datos > registro.py > ...
1  # Ejercicio validación de usuarios
2
```

2. Implementa una función llamada **validar_datos** (nombre, edad, correo) que cumpla con los siguientes requisitos:
 - a. Verifica que **nombre** sea una cadena de texto no vacía.
 - b. Verifica que **edad** sea un número entero mayor que cero.
 - c. Verifica que **correo** contenga un símbolo @.

```
def validar_datos():
    print("\n===== INICIANDO =====")
    logging.info("Validando datos...\n")

    while True:
        try:
            Nombre = input("Ingresa tu nombre: ")

            if Nombre.strip() == "":
                raise TypeError("Entrada invalida, el nombre no puede estar vacío")
```

```
while True:
    try:
        Edad = int(input("Ahora ingresa tu edad: "))
        if Edad < 0:
            print("---- Error: Ingrese un valor diferente ----")
            validar_datos()
```

```
while True:
    try:
        Correo = input("Ingresa tu correo: ")
        v_email(Correo)
```

3. Implementa el uso de **try**, **except** y **finally** para:
 - a. Capturar errores de tipo (TypeError, ValueError, etc.).
 - b. Informar al usuario si los datos fueron registrados correctamente o no.
 - c. Asegurar que un mensaje de cierre siempre se muestre con finally.

```
while True:
    try:
        Correo = input("Ingresa tu correo: ")
        v_email(Correo)
    except EmailNotValidError:
        logging.warning(f"Error en el correo: un correo debe tener el signo @")
        print("El correo ingresado es invalido, revisa por favor")
    except Exception as e:
        logging.error(f"Error en el correo: {e}")
        print(type(e).__name__)
    else:
        break

    finally:
        logging.info("---- Registro finalizado ----")
```

4. Incluye al menos 3 excepciones diferentes.
 - a. Por ejemplo: ValueError, TypeError, ZeroDivisionError (si decides extender con otra validación como división entre edades, etc.).

```

while True:
    try:
        Edad = int(input("Ahora ingresa tu edad: "))
        if Edad < 0:
            print("---- Error: Ingrese un valor diferente ----")
            validar_datos()

        validar_edad = 1/Edad

    except ZeroDivisionError:
        logging.warning(f"Error en edad: ")
        print("---- La edad no puede ser 0 ----")
    except ValueError:
        logging.warning(f"Error en edad: {e}")
        print("---- Ingrese un numero por favor ----")
    except Exception as e:
        logging.warning(f"Error en edad: {e}")
        print(type(e).__name__)
    else:
        break

```

5. Agrega una función de prueba (probar_validaciones()) que llame a validar_datos.

```

def probar_validaciones():
    validar_datos()

probar_validaciones()

```

6. Incluye al menos dos mensajes de depuración impresos con print() o usando el módulo logging.

```

print("\n===== INICIANDO =====")
logging.info("Validando datos...\n")

```

```

except ValueError as e:
    logging.warning(f"Error en nombre: {e}")
    print(f"{e}, favor de revisar")

```

```

finally:
    logging.info("---- Registro finalizado ----")

```

7. Salida final esperada.

- Imprimir Datos capturados sin errores.
- Imprimir Datos capturados con algo incorrecto:

SALIDA EN VS sin errores.

```
PS C:\Users\hurta\Documents\2026\curso-pyinter> & C:\Users\hurta\AppData\Local\Programs\Python\Python313\python.exe
c:/Users/hurta/Documents/2026/curso-pyinter/Ejercicio_U4_Validación_de_Datos/registro.py

===== INICIANDO =====
INFO: Validando datos...

Ingresa tu nombre: Pedro
Ahora ingresa tu edad: 41
Ingresa tu correo: pedrohl@email.com
INFO: ---- Registro finalizado ----

===== DATOS CAPTURADOS =====

Nombre del usuario: Pedro
Edad del usuario: 41 años
Correo del usuario: pedrohl@email.com
```

SALIDA EN VS con errores.

```
PS C:\Users\hurta\Documents\2026\curso-pyinter> & C:\Users\hurta\AppData\Local\Programs\Python\Python313\python.exe
c:/Users/hurta/Documents/2026/curso-pyinter/Ejercicio_U4_Validación_de_Datos/registro.py

===== INICIANDO =====
INFO: Validando datos...

Ingresa tu nombre:
WARNING: Error en nombre:
Hay algún error en el nombre, revisa por favor
Ingresa tu nombre: Pedro
Ahora ingresa tu edad: 0
WARNING: Error en edad:
---- La edad no puede ser 0 ----
Ahora ingresa tu edad: 41
Ingresa tu correo: pedrohl_email.com
WARNING: Error en el correo: un correo debe tener el signo @
El correo ingresado es invalido, revisa por favor
INFO: ---- Registro finalizado ----
Ingresa tu correo:
WARNING: Error en el correo: un correo debe tener el signo @
El correo ingresado es invalido, revisa por favor
INFO: ---- Registro finalizado ----
Ingresa tu correo: pedrohl@email.com
INFO: ---- Registro finalizado ----

===== DATOS CAPTURADOS =====

Nombre del usuario: Pedro
Edad del usuario: 41 años
Correo del usuario: pedrohl@email.com
```

Conclusiones.

- En conclusión, este ejercicio nos permitió reforzar los conocimientos teóricos de la validación de datos, implementando el manejo de excepciones para crear programas más seguros, estables y fluidos, evitando los cortes bruscos al ejecutar un programa. Además, el uso de try, except y finally nos permite manejar distintos tipos de errores, mientras que los mensajes de depuración ayudan a entender y mejorar el funcionamiento del código.